

Pertemuan 6

Pembersihan dan pra-pemrosesan data

Apa itu missing value

Missing Value adalah hilangnya beberapa data yang telah diperoleh.

Missing value erat kaitannya dalam proses perselisihan data (data wrangling) sebelum nantinya akan dilakukan analisis dan prediksi data.

Apa itu missing value (lanjut ..)

Data wrangling merupakan kegiatan penyeragaman data atau pembersihan data (cleaning data) dari data kotor (mentah) menjadi data yang nantinya siap digunakan untuk analisis.

Data kotor (mentah) yang dimaksud adalah data yang terindikasi masih terdapat ketidakseragaman format, muncul missing values pada data, dan masih juga ditemukan adanya tambahan sufiks, prefiks dan lain-lain. Biasanya, seorang data scientist menghabiskan 60% waktunya dalam melakukan proses ini. Karena fakta menunjukkan bahwa 75% data yang dimiliki oleh perusahaan adalah data kotor

Jenis Missing Value

Missing at Random (MAR).

Jika mekanisme data hilang terdistribusi secara acak untuk sebagian unit observasi. Dengan kata lain, Missing at Random (MAR), berarti terjadinya missing data hanya berkaitan dengan variabel respon/pengamatan.

Contohnya seseorang yang memiliki rasa was-was yang tinggi cenderung tidak akan melaporkan pendapatan mereka, rasa was was akan berhubungan pada pelaporan pendapatan

Jenis Missing Value (lanjut..)

Missing completely at random (MCAR)

Jika mekanisme data hilang yang terdistribusi secara acak untuk seluruh unit observasi. Dengan kata lain, **Missing Completely at Random (MCAR)** yang berarti bahwa terjadinya missing data tidak berkaitan dengan nilai semua variabel, apakah itu variabel dengan missing values atau dengan variabel pengamatan. Hal ini berarti missing data terjadi secara acak.

Jenis Missing Value (Lanjut..)

Missing Not at Random (MNAR)

Mekanisme data hilang yang tidak terdistribusi secara random. Dengan kata lain, Missingness Is Non-Ignorable bahwa terjadinya missing data pada suatu variabel berkaitan dengan variabel itu sendiri, sehingga ini tidak bisa diprediksi dari variabel lain pada suatu dataset.

Faktor Penyebab Missing Value

Faktor non sampling error yang dimaksud adalah interviewer recording error, respondent inability error, dan respondent unwillingness error.

Interviewer recording error

terjadi akibat kelalaian petugas pengumpul data (pewawancara), misalnya ada sejumlah pertanyaan yang terlewatkan.

Respondent inability error

terjadi akibat ketidakmampuan responden dalam memberikan jawaban akurat, misalnya karena tidak memahami pertanyaan, bosan atau kelelahan (respondent fatigue) akhirnya responden mengosongkan sejumlah pertanyaan atau berhenti mengisi kuesioner di tengah jalan.

Faktor Penyebab Missing Value (Lanjut..)

Unwillingness respondent error

terjadi karena responden tidak berkenan memberikan jawaban yang akurat, misalnya pertanyaan soal penghasilan, usia, berat badan, pengalaman melakukan pelanggaran hukum, dll. Seperti halnya pada respondent inability error, responden bisa mengosongkan jawaban atau menghentikan proses pengisian kuesioner.

Cara Penangan Missing Value

1. Mengabaikan dan membuang missing data

Contoh metode yang sering digunakan pada kategori ini adalah metode Listwise deletion dan Pairwise deletion

2. Estimasi parameter

Contohnya algoritma Expectation-Maximization (EM Algorithm) yang digunakan untuk mengestimasi parameter dari missing data.

3. Imputasi

Proses pengisian atau penggantian nilai-nilai yang hilang (missing values) pada sekumpulan data (dataset) dengan nilai-nilai yang mungkin (plausible values) berdasarkan informasi yang didapatkan pada dataset tersebut

Mengidentifikasi Missing Values

Pada python Missing value dapat diidentifikasi dengan perintah :

```
missing_data = df.isnull().sum()  
print(missing_data)
```

Mengabaikan dan membuang Missing Value

Listwise Deletion

Listwise deletion adalah metode di mana seluruh baris data yang memiliki missing values dihapus dari dataset

```
[2] import pandas as pd

#Contoh DataFrame dengan Missing Values
data={
    'A' : [1, 2 ,None,4,5],
    'B' : [None, 2 ,3, 4,None],
    'C' : [1, None , 3,4,5],
}

df=pd.DataFrame(data)

df_listwise=df.dropna()

print("Data Setelah List Wise Deletion : ")
print(df_listwise)
```

Data Setelah List Wise Deletion :

	A	B	C
3	4.0	4.0	4.0

Mengabaikan dan membuang Missing value (lanjut)

Pairwise Deletion

Pairwise deletion menghapus missing values hanya pada pasangan variabel yang terlibat dalam perhitungan tertentu. Dalam Pandas, ini tidak langsung didukung, tetapi bisa diterapkan secara manual untuk operasi tertentu seperti korelasi

```
#Contoh DataFrame dengan Missing Values
data={
    'A' : [1, 2 ,None,4,5],
    'B' : [None, 2 ,3, 4,None],
    'C' : [1, None , 3,4,5],
}

df=pd.DataFrame(data)

korelasi=df[['A','B']].dropna().corr()

print("Korelasi Pairwise Deletion antara A dan B : ")
print(korelasi)
```

```
⇒ Korelasi Pairwise Deletion antara A dan B :
      A      B
A  1.0  1.0
B  1.0  1.0
```

Mengabaikan dan membuang Missing value (lanjut)

Estimasi Parameter: Expectation-Maximization (EM Algorithm)

EM algorithm biasanya diterapkan dalam kasus yang lebih kompleks seperti clustering dengan Gaussian Mixture Models (GMM). Berikut adalah contoh sederhana dengan sklearn untuk estimasi parameter menggunakan EM dalam konteks clustering:

```
[7] import numpy as np
    from sklearn.mixture import GaussianMixture
    from sklearn.impute import SimpleImputer

data=np.array([
    [1,2],
    [2,3],
    [None,6],
    [6,8],
    [None,None],
    [3,4]
])

#Mengisi missing value dengan nilai rata-rata
imputer=SimpleImputer(strategy='mean')
data_imputed=imputer.fit_transform(data)

#Menerapkan algoritma EM dengan GMM
gmm=GaussianMixture(n_components=2,max_iter=100, random_state=42)
gmm.fit(data_imputed)

#Estimasi parameter (mean dan covarian matrix)
print("Rerata atau mean dari tiap komponen Gaussian :")
print(gmm.means_)
print("\nKovarian Matrix dari tiap komponen Gaussian :")
print(gmm.covariances_)
```

Mengabaikan dan membuang Missing value (lanjut)

Imputasi data

Merupakan salah satu solusi untuk mengganti suatu missing value dengan suatu nilai pengganti yang didapatkan dari teknik khusus tertentu, misalnya imputasi menggunakan nilai mean, nilai median, dan lain sebagainya.

```
#Contoh DataFrame dengan Missing Values
data={
    'A' :[1, 2 ,None,4,5],
    'B' :[None, 2 ,3, 4,None],
    'C' :[1, None , 3,4,5],
}

df=pd.DataFrame(data)

#Mengganti missing value dengan nilai rata-rata kolom
imputer=SimpleImputer(strategy='mean')
df_imputed=pd.DataFrame(imputer.fit_transform(df),columns=df.columns)

print("Data setelah imputasi dengan rata-rata ")
print(df_imputed)
```

```
➡ Data setelah imputasi dengan rata-rata
```

	A	B	C
0	1.0	3.0	1.00
1	2.0	2.0	3.25
2	3.0	3.0	3.00
3	4.0	4.0	4.00
4	5.0	3.0	5.00

Outliers

Outlier atau nilai pencilon seperti kalimat atau keterangan yang menjadi kambing hitam dalam data kita karena merupakan titik data yang berbeda (misalnya bernilai jauh lebih besar atau jauh lebih kecil) secara signifikan dari pengamatan lain dalam kumpulan data secara keseluruhan. Hal ini dapat terjadi karena alasan seperti kesalahan pengukuran, kesalahan entri data, atau bisa juga karena pengamatan yang valid namun ekstrem

Outliers (lanjut..)

Salah satu cara untuk mengidentifikasi keberadaan *outlier* pada data adalah dengan membuat *box plot*, seperti yang dicontohkan pada perintah python berikut ini:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Membuat dataset dengan nilai pencilan
np.random.seed(42)
data = {
    'A': np.random.normal(loc=50, scale=10, size=100),
    'B': np.random.normal(loc=50, scale=10, size=100),
    'C': np.random.normal(loc=50, scale=10, size=100)
}

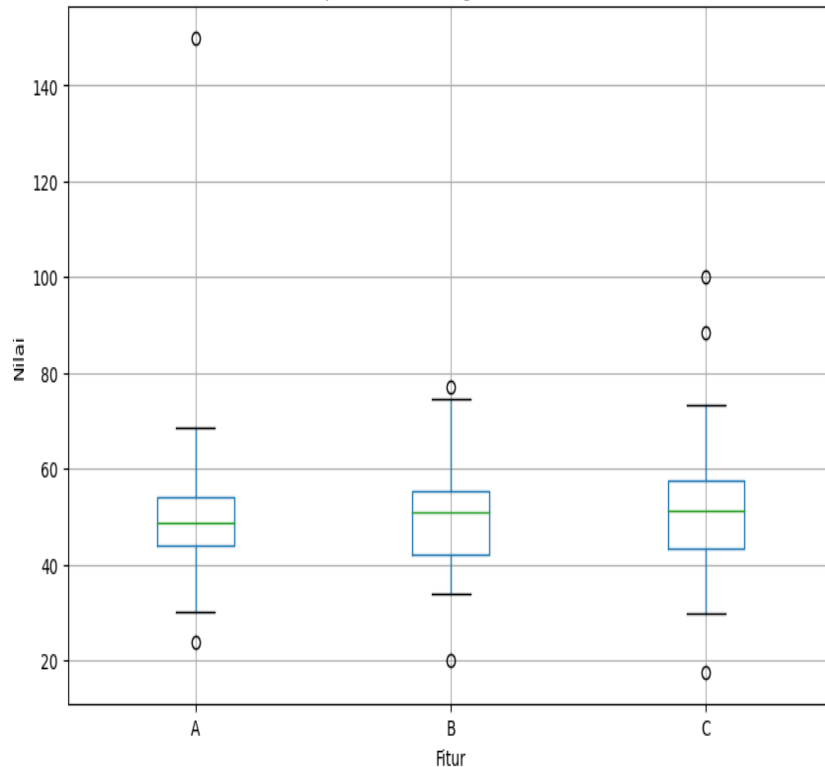
# Menambahkan nilai pencilan ke dalam dataset
data['A'][0] = 150
data['B'][10] = 20
data['C'][5] = 100

# Membuat DataFrame
df = pd.DataFrame(data)

# Membuat boxplot
plt.figure(figsize=(10, 6))
df.boxplot()
plt.title('Boxplot Dataset dengan Nilai Pencilan')
plt.ylabel('Nilai')
plt.xlabel('Fitur')
plt.grid(True)
plt.show()
```

Box Plot

Boxplot Dataset dengan Nilai Pencilan



Hasil Boxplot dari script python sebelumnya

Boxplot adalah grafik yang memberikan indikasi visual mengenai bagaimana nilai persentil ke-25, persentil ke-50, persentil ke-75, minimum, maksimum, dan outlier dari suatu kumpulan data disebarakan dan dibandingkan satu sama lain.

Duplikasi data

Data duplikat dan redundan merupakan dua masalah yang berbeda, yang dapat muncul pada sebuah dataset. Duplikat adalah catatan yang berulang dalam data. Hal ini dapat membiaskan analisis dan menghasilkan kesimpulan yang salah. Sedangkan data redundan adalah data yang tidak menambahkan informasi baru. Meskipun tidak berbahaya seperti duplikat, data ini dapat memperlambat komputasi dan menghabiskan ruang penyimpanan yang tidak perlu.

Penanganan Duplikasi Data

Penyebab Duplikasi data:

- Data Entry Manual
- Integrasi Data dari Sumber Berbeda
- Kesalahan Sistem

Dampak Negatif Duplikasi Data:

- Analisis yang Tidak Akurat
- Kinerja Model yang Buruk
- Konsumsi Sumber Daya

Penanganan Duplikasi Data (lanjut)

Penghapusan Duplikasi

Data yang duplikat dapat mengganggu analisis dan menyebabkan kesalahan interpretasi. Menghapus entri yang duplikat membantu menghindari redundansi

```
import pandas as pd

# Contoh DataFrame dengan duplikasi
data = {
    'A': [1, 2, 2, 3, 4, 4, 5],
    'B': ['a', 'b', 'b', 'c', 'd', 'd', 'e'],
    'C': [10, 20, 20, 30, 40, 40, 50]
}
df = pd.DataFrame(data)

# Mencari baris duplikat
duplicate_rows = df.duplicated()

# Menampilkan baris duplikat
print("Baris Duplikat:")
print(df[duplicate_rows])

# Menghapus baris duplikat
df_no_duplicates = df.drop_duplicates()

print("Data setelah penghapusan duplikat:")
print(df_no_duplicates)
```

Penanganan Duplikasi Data (lanjut)

Data deduplikasi

Data deduplikasi merupakan sebuah Teknik untuk mengeliminasi duplikasi data, dengan penggabungan 2 data

```
import pandas as pd

# Contoh DataFrame 1
data1 = {
    'ID': [1, 2, 3, 4],
    'Nama': ['Alice', 'Bob', 'Charlie', 'David'],
    'Kota': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
df1 = pd.DataFrame(data1)

# Contoh DataFrame 2
data2 = {
    'ID': [3, 4, 5, 6],
    'Nama': ['Charlie', 'David', 'Emily', 'Frank'],
    'Kota': ['Chicago', 'Houston', 'San Francisco', 'Seattle']
}
df2 = pd.DataFrame(data2)

# Menggabungkan DataFrame menggunakan merge() dengan 'ID' sebagai kunci
merged_df = pd.merge(df1, df2, on='ID', how='outer')

# Menampilkan DataFrame yang telah digabungkan
print("DataFrame yang telah digabungkan:")
print(merged_df)

# Menghapus baris duplikat berdasarkan kolom 'ID'
deduplicated_df = merged_df.drop_duplicates(subset=['ID'])

# Menampilkan DataFrame setelah deduplikasi
print("\nDataFrame setelah deduplikasi:")
print(deduplicated_df)
```